



INTRODUCCIÓN A LA ROBÓTICA E1505

NAVEGACIÓN I



INTRODUCCIÓN A LA ROBÓTICA

Índice:

- 1 Navegación
- 2 Navegación sin mapa
- 3 Navegación con mapa
- 4 Navegación con mapa planificada



INTRODUCCIÓN A LA ROBÓTICA

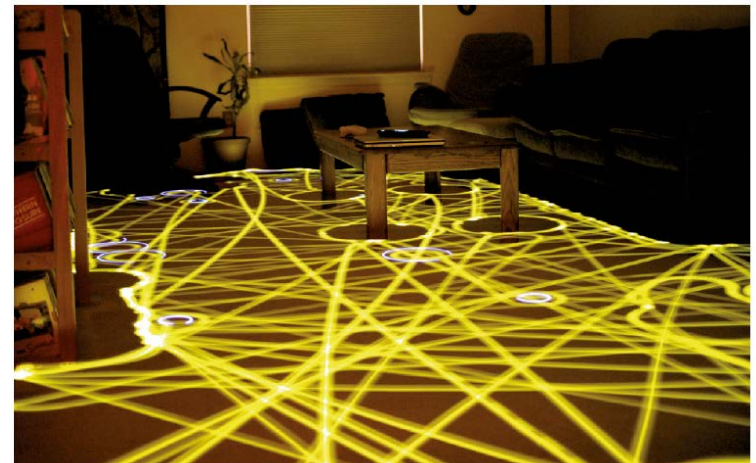
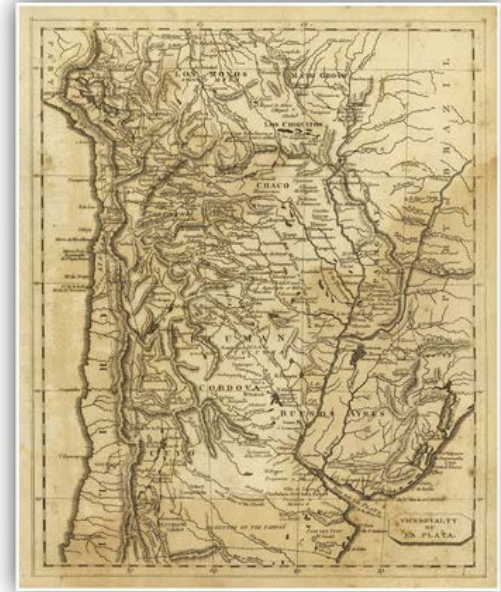
Índice:

- 1 Navegación**
- 2 Navegación sin mapa
- 3 Navegación con mapa
- 4 Navegación con mapa planificada



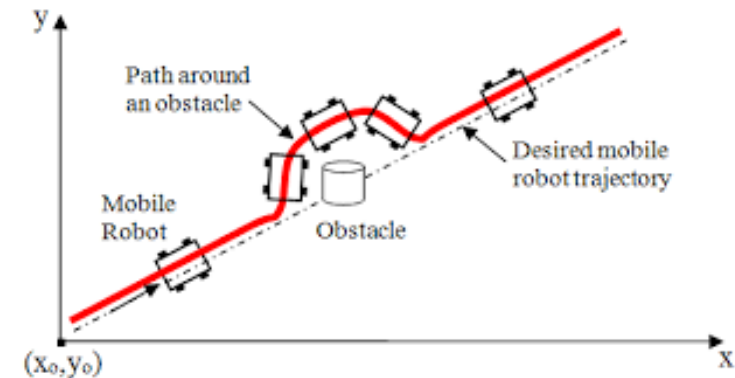
Navegación.

- Es el proceso de direccionar un vehículo para alcanzar un destino previsto. - IEEE Estándar 172-1983 .
- Enfoque humano, crear un mapa y colocar marcas de guiado
- Enfoque robot:
 - **Navegar sin mapa.** Ej. seguir una luz, una línea en el suelo, camino aleatorio...
 - **Navegar con mapa.** Planificación de movimiento – robots más complejos.



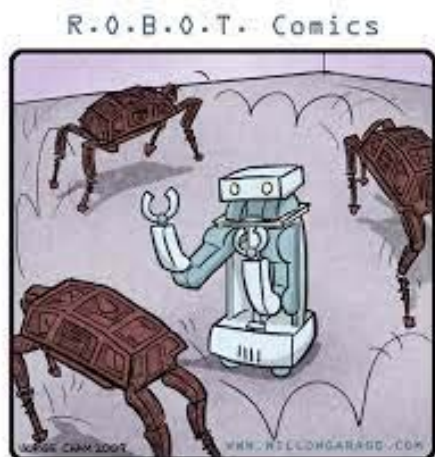
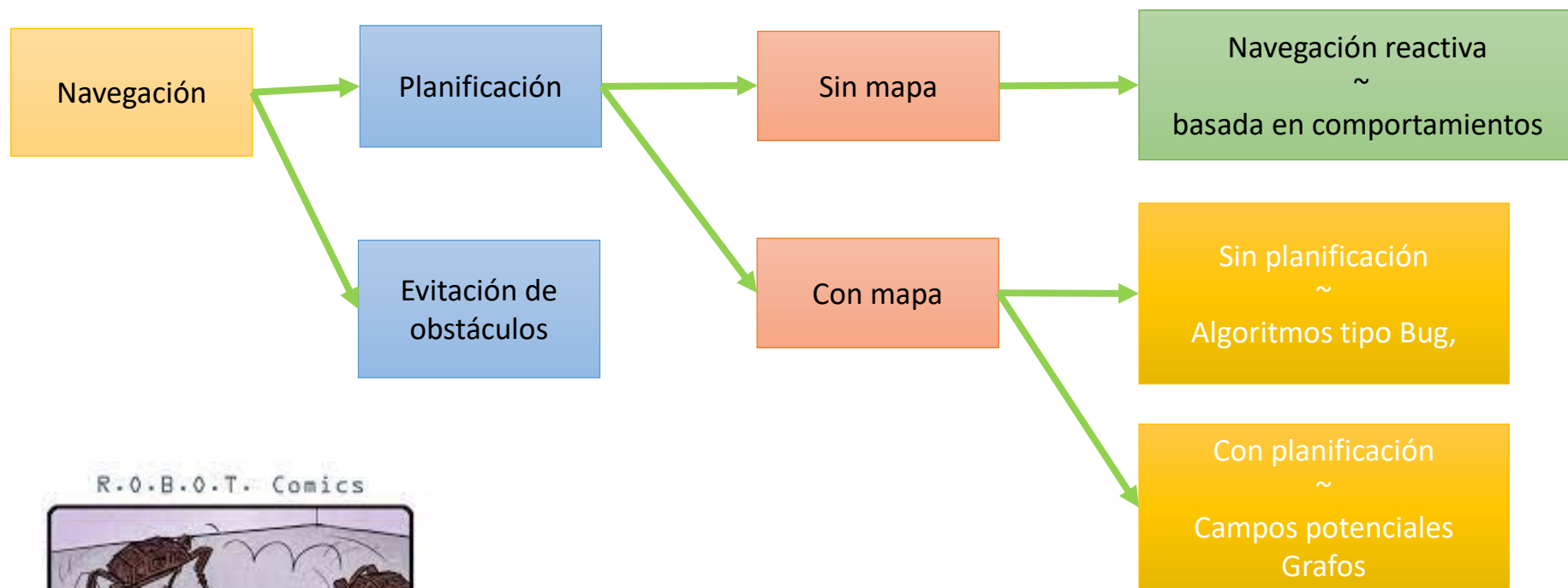
Navegación.

- El proceso de navegación implica la resolución de dos problemas:
 - Dado un mapa y un punto de destino, la **planificación de camino (path planning)** identificara una trayectoria (o camino) que lograra llevar al robot a la posición destino. (Podemos verlo como un problema estratégico – el robot decide objetivos a largo plazo).
 - Evitación de obstáculos (obstacle avoidance)**: implica modificar la trayectoria del robot para evitar colisiones.
 - Un enfoque extremo de ambos problemas es llamado **planificación integrada y ejecución (integrated planning and execution)**.

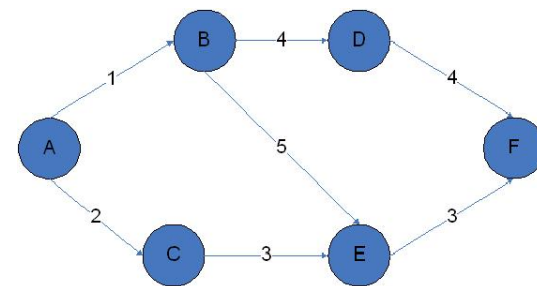
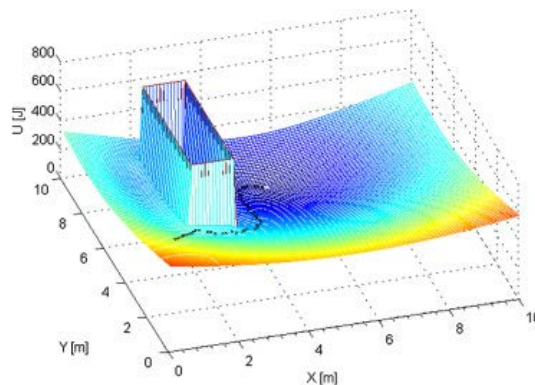




IR: 1 - NAVEGACIÓN



"SIT, BOY, SIT! SIT, I SAY,
SI... OH, FORGET IT."





INTRODUCCIÓN A LA ROBÓTICA

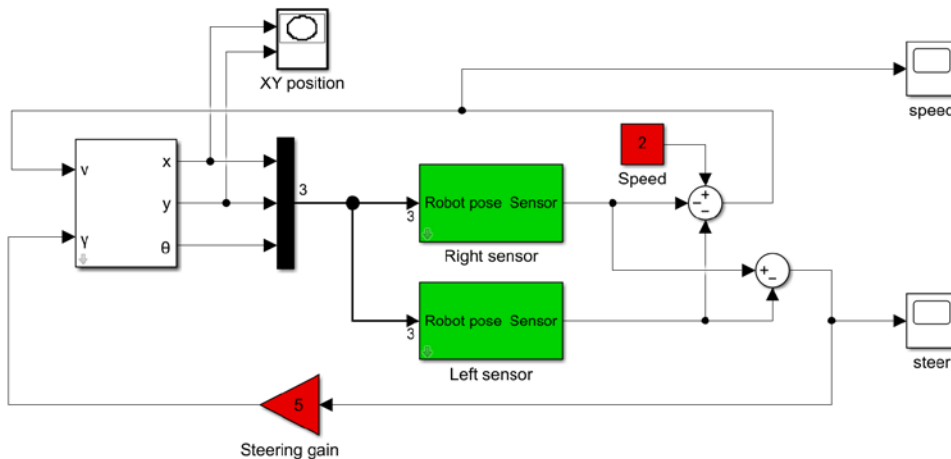
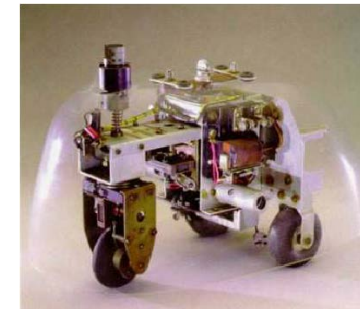
Índice:

- 1 Navegación
- 2 Navegación sin mapa**
- 3 Navegación con mapa
- 4 Navegación con mapa planificada

Navegación reactiva.

- Robots conocidos como vehículo de Braitenberg, realizan una conexión directa entre sensores y motores (no hay representación del ambiente ni planificación).

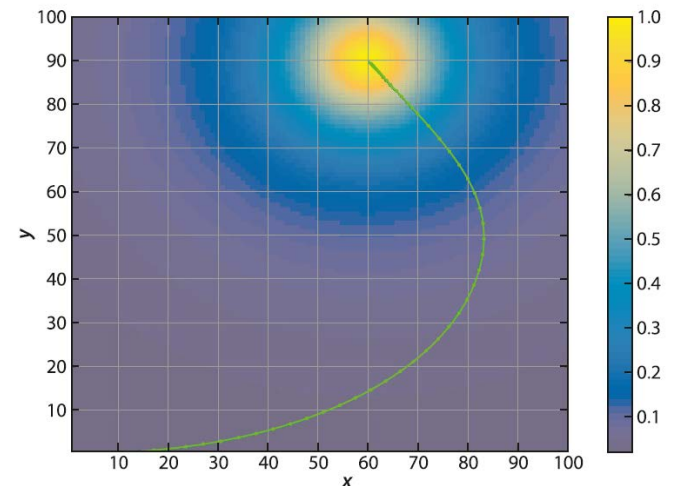
Grey Walter robot 1948



$$v = 2 - s_R - s_L$$

$$\gamma = k(s_R - s_L)$$

```
sl_braitenberg
sim('sl_braitenberg')
```





INTRODUCCIÓN A LA ROBÓTICA

Índice:

1

Navegación

2

Navegación sin mapa

3

Navegación con mapa

4

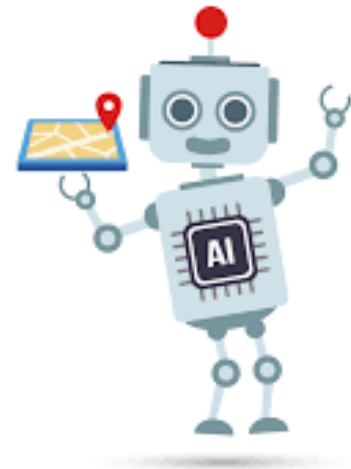
Navegación con mapa planificada



IR: 3 – NAVEGACIÓN CON MAPA

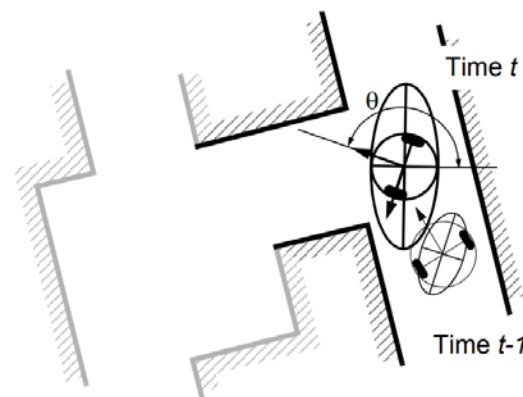
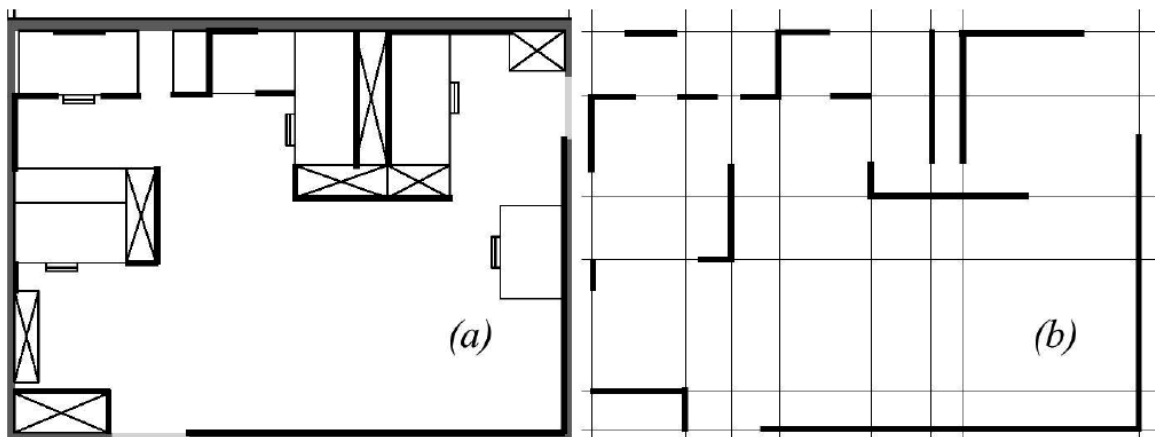
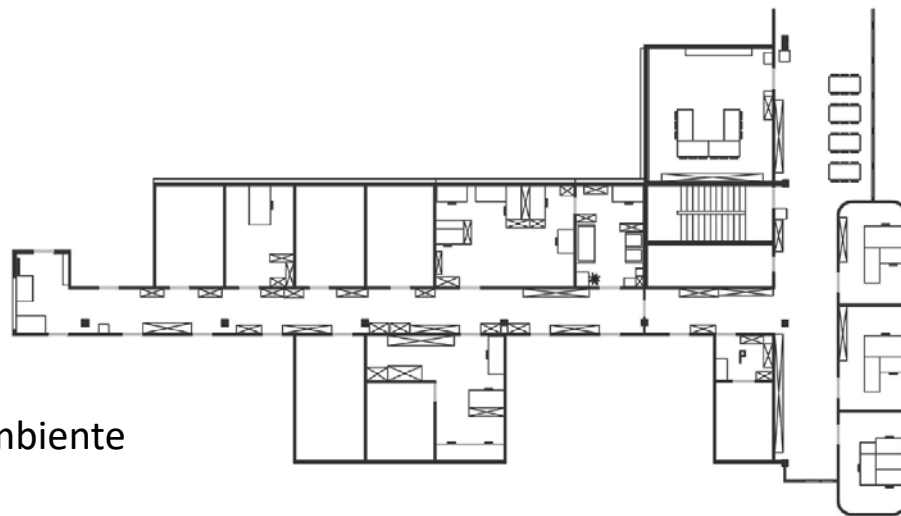
Mapas.

- Forma de representación del ambiente de trabajo
- La elección del tipo de mapa puede ser visto como un problema doble:
 - **Representar el ambiente** en el que el robot se mueve (que tan exacto lo necesito?)
 - **Representar la posición del robot** en el ambiente (que tan exacto lo necesito?)
- Tres puntos fundamentales deben tenerse en cuenta al elegir el tipo de mapa a utilizar:
 - La **precisión del mapa**, debe coincidir con la precisión que el robot necesita para completar sus objetivos.
 - La **precisión del mapa y características representadas por el mismo**, deben coincidir con la precisión y tipo de datos obtenidos de los sensores del robot.
 - La **complejidad en la representación del mapa** impacta directamente en la complejidad de cómputo del mapeo, localización y navegación.



Tipos de Mapas.

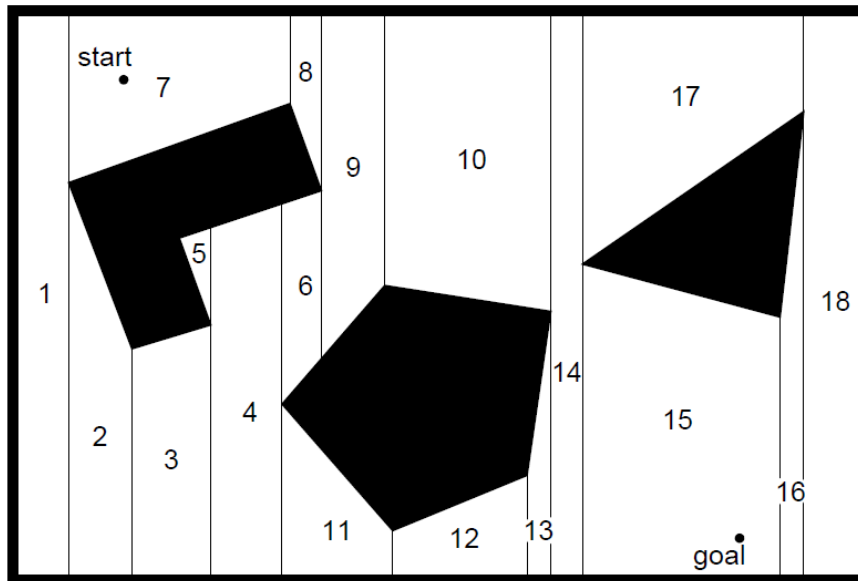
- **Representaciones continuas**
 - Representación mediante polígonos.
 - Representación mediante líneas.
- Características:
 - Suposición del mundo cerrado
 - Gran precisión en la representación del ambiente
 - Computacionalmente costosos.





Tipos de Mapas.

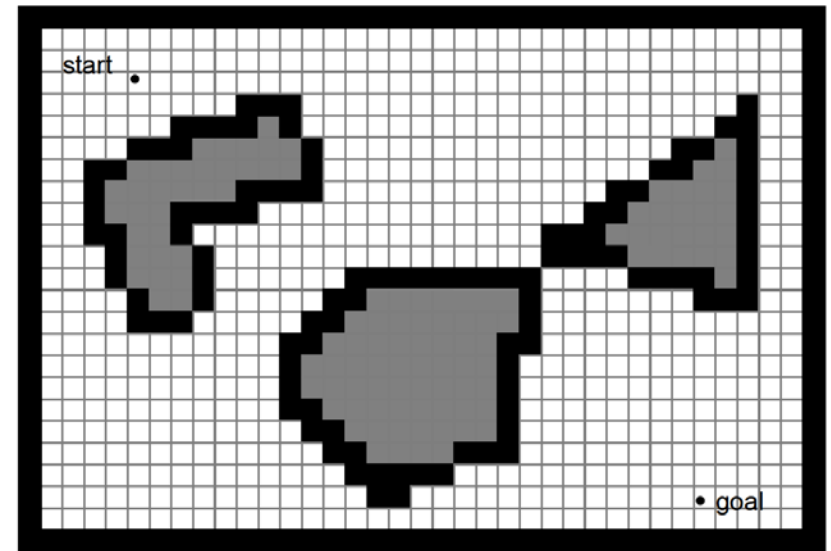
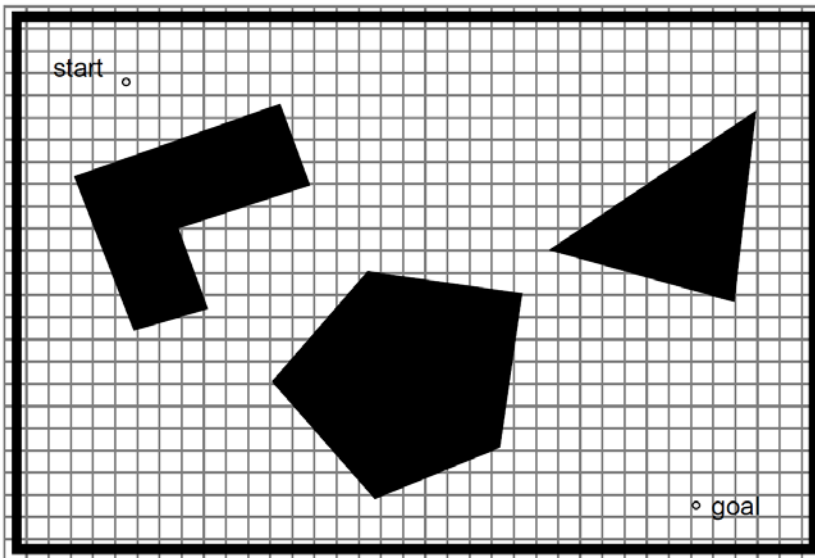
- **Representaciones discretas (estrategias de descomposición)**
 - Descomposición en celdas exactas (exact cell decomposition).
 - Descomposición fija (fixed decomposition) -> grillas de ocupación (occupancy grid)
 - Descomposición adaptativa (adaptative cell decomposition).



**Descomposición en
celdas exacta**

Tipos de Mapas.

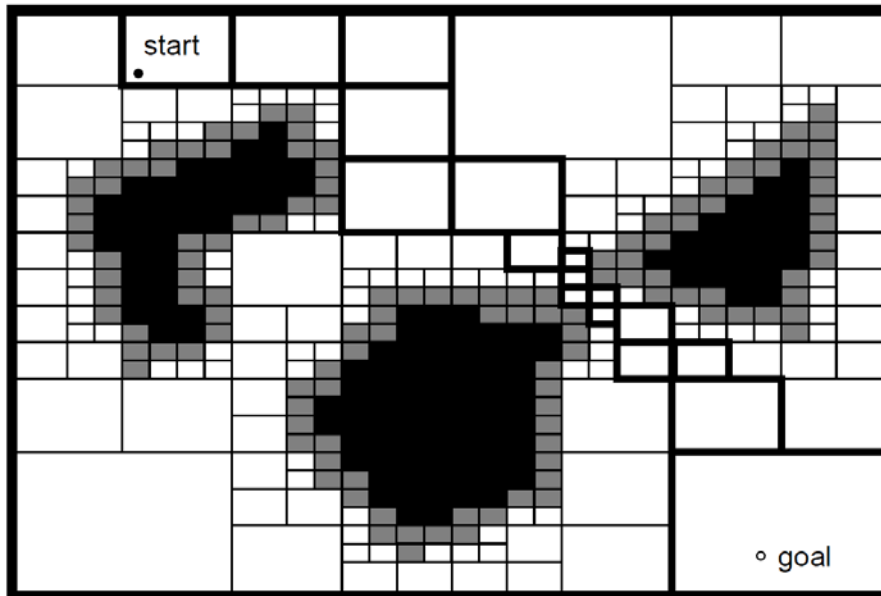
- Representaciones discretas (estrategias de descomposición)
 - Descomposición en celdas exactas (exact cell decomposition).
 - Descomposición fija (fixed decomposition) -> **grillas de ocupación** (occupancy grid)
 - Descomposición adaptativa (adaptative cell decomposition).



Descomposición en celdas fijas

Tipos de Mapas.

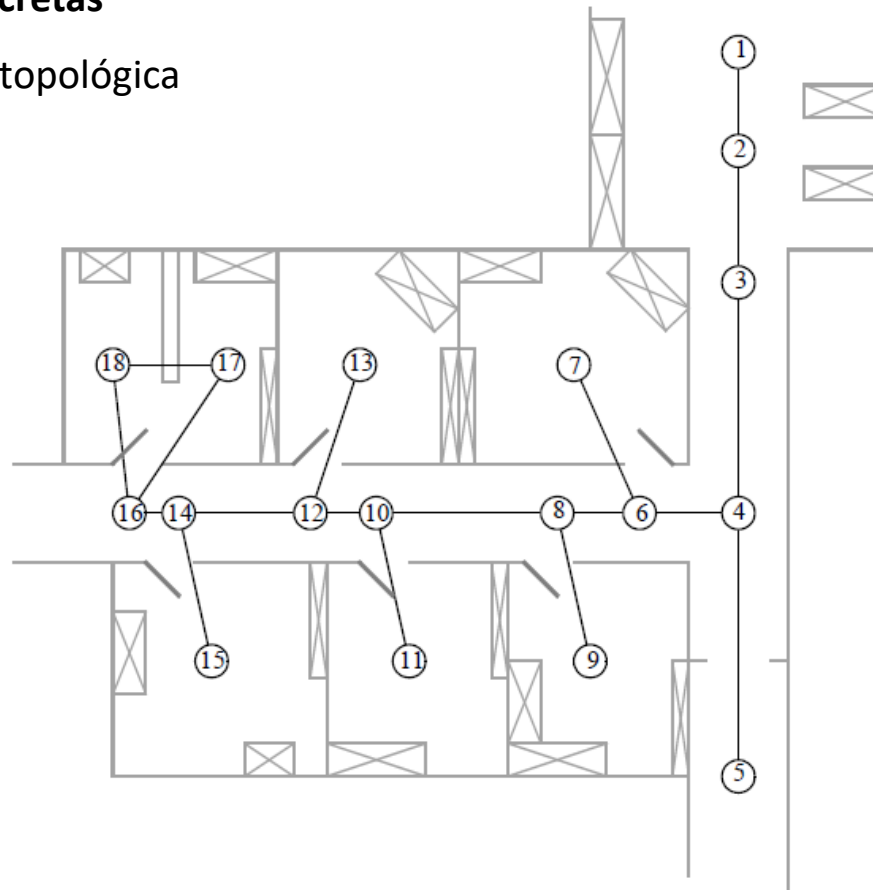
- Representaciones discretas (estrategias de descomposición)
 - Descomposición en celdas exactas (exact cell decomposition).
 - Descomposición fija (fixed decomposition) -> **grillas de ocupación** (occupancy grid)
 - Descomposición adaptativa (adaptative cell decomposition).



Descomposición adaptativa

Tipos de Mapas.

- Representaciones discretas
 - Representación topológica



Representación topológica



Algoritmos bug (insecto).

- Realizan una búsqueda del objetivo en presencia de áreas ocupadas u obstáculos.
- Son similares al enfoque reactivo, pero agregan una máquina de estados para relacionar los sensores y los actuadores -> **agregan memoria**
- Se realizan algunas suposiciones:
 - El robot trabaja en mundo posible de modelizar como una grilla de ocupación.
 - El robot ocupa un casillero de la grilla de ocupación.
 - El robot es omnidireccional (puede moverse a cualquiera de sus 8 casillas vecinas).
 - Es capaz de determinar su posición en el plano **(Problema no trivial!)**



IR: 3 – NAVEGACIÓN CON MAPA

Algoritmos bug 1.

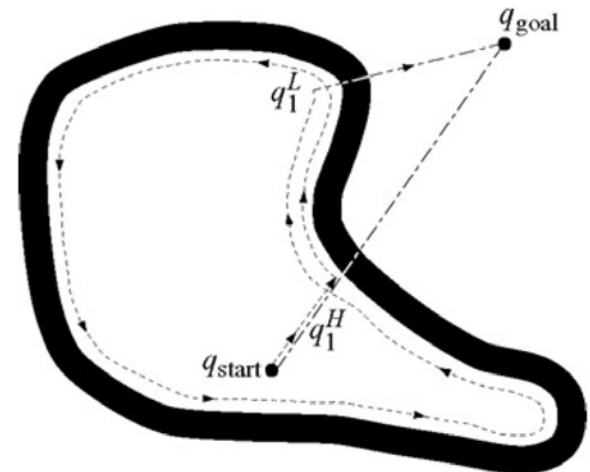
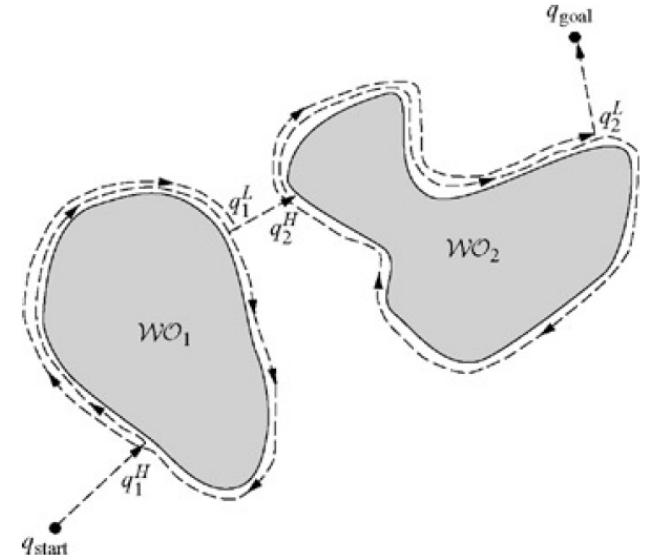
Input: A point robot with a tactile sensor

Output: A path to the q_{goal} or a conclusion no such path exists

```
1: while Forever do
2:   repeat
3:     From  $q_{i-1}^L$ , move toward  $q_{\text{goal}}$ .
4:   until  $q_{\text{goal}}$  is reached or an obstacle is encountered at  $q_i^H$ 
5:   if Goal is reached then
6:     Exit.
7:   end if
8:   repeat
9:     Follow the obstacle boundary.
10:  until  $q_{\text{goal}}$  is reached or  $q_i^H$  is re-encountered.
11:  Determine the point  $q_i^L$  on the perimeter that has the shortest distance to the goal.
12:  Go to  $q_i^L$ .
13:  if the robot were to move toward the goal then
14:    Conclude  $q_{\text{goal}}$  is not reachable and exit.
15:  end if
16: end while
```

Comportamientos:

1. Movimiento a objetivo
2. Seguimiento de frontera



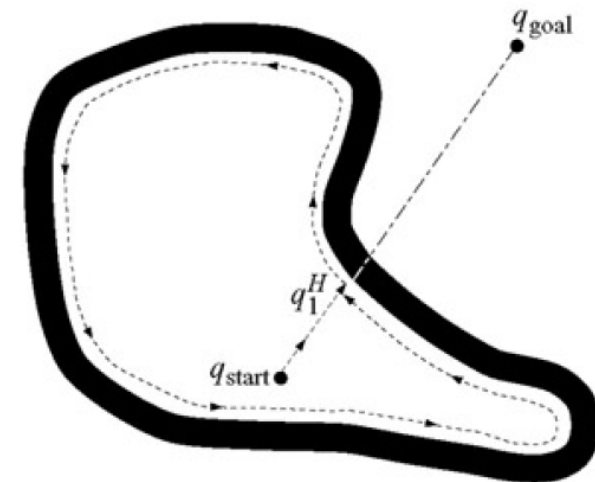
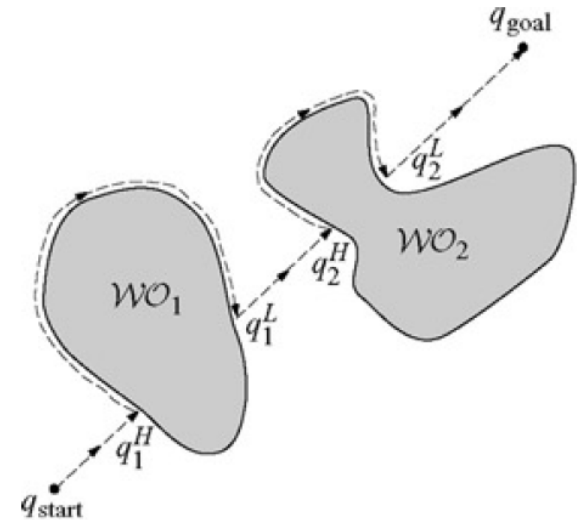
Algoritmos bug 2.

Input: A point robot with a tactile sensor

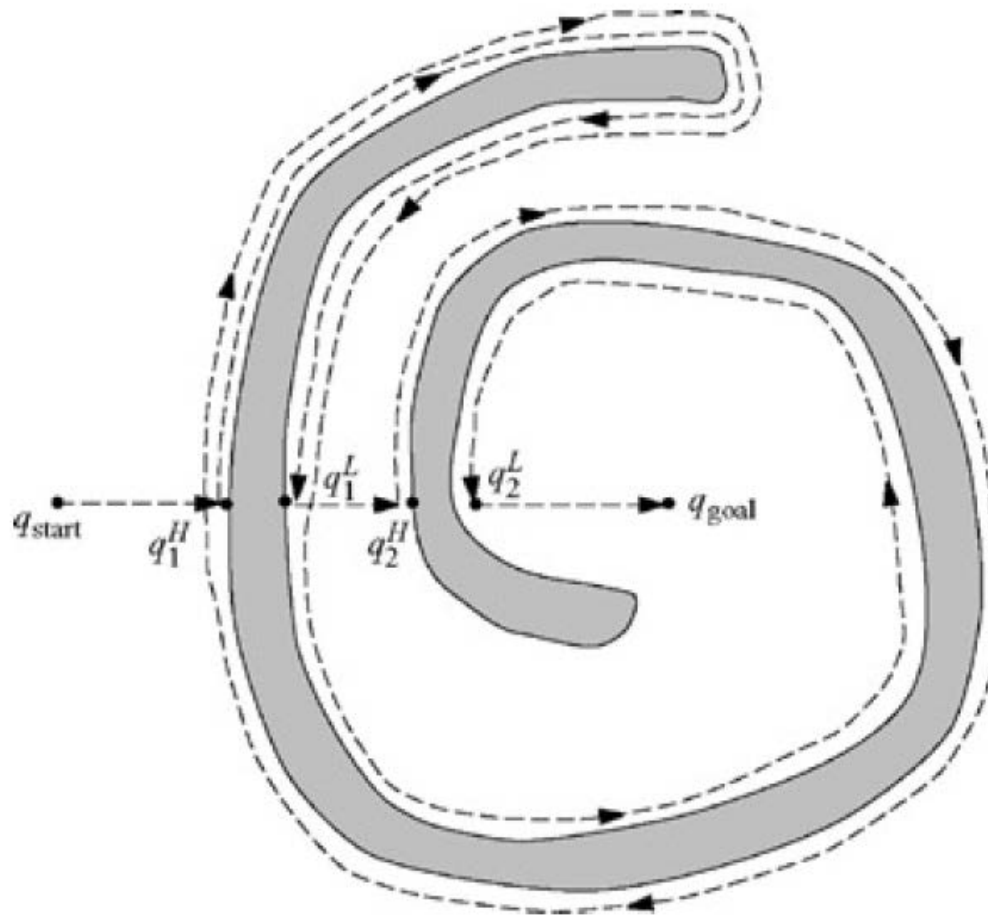
Output: A path to q_{goal} or a conclusion no such path exists

```

1: while True do
2:   repeat
3:     From  $q_{i-1}^L$ , move toward  $q_{\text{goal}}$  along  $m$ -line.
4:   until
 $q_{\text{goal}}$  is reached or
an obstacle is encountered at hit point  $q_i^H$ .
5:   Turn left (or right).
6:   repeat
7:     Follow boundary
8:   until
 $q_{\text{goal}}$  is reached or
 $q_i^H$  is re-encountered or
 $m$ -line is re-encountered at a point  $m$  such that
12:      $m \neq q_i^H$  (robot did not reach the hit point),
13:      $d(m, q_{\text{goal}}) < d(m, q_i^H)$  (robot is closer), and
14:     If robot moves toward goal, it would not hit the obstacle
15:   Let  $q_{i+1}^L = m$ 
16:   Increment  $i$ 
17: end while
  
```



Algoritmos bug 2.





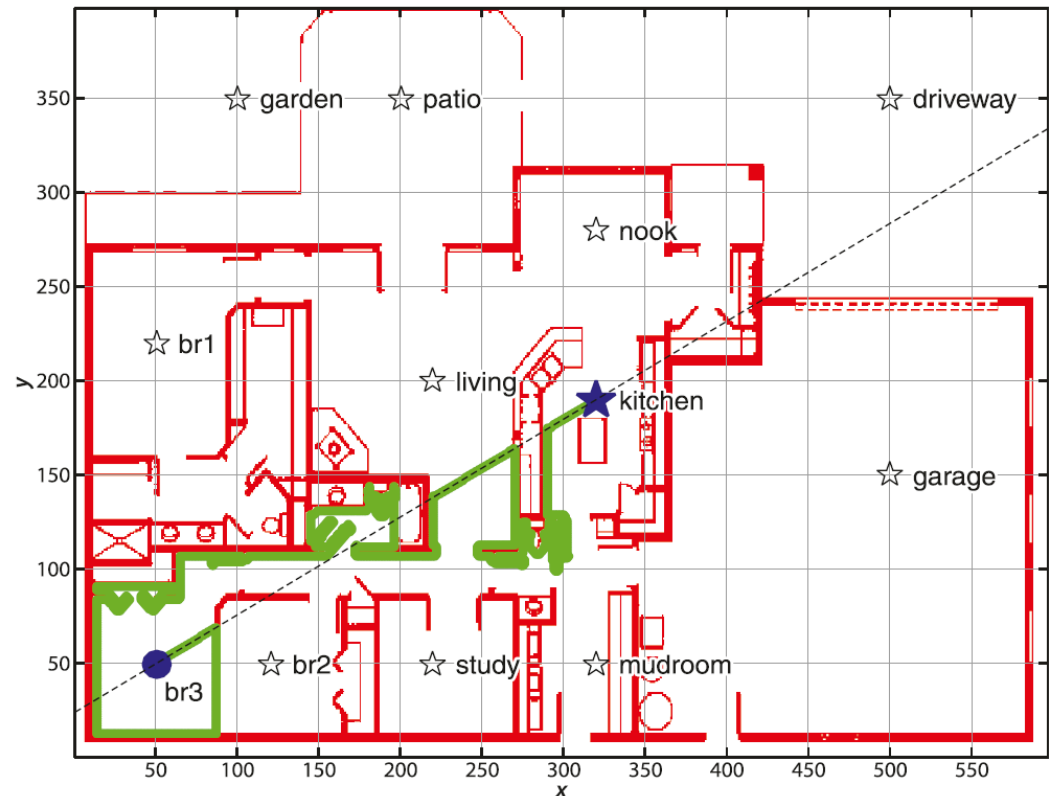
IR: 3 – NAVEGACIÓN CON MAPA

Algoritmos bug 2.

```
close all  
clear all  
load house place  
bug=Bug2(house);  
bug.plot();  
bug.query(place.br3,  
place.kitchen, 'animate')
```

Ver que este algoritmo estrictamente no
necesita un mapa y puede basarse en
medidas locales al robot.

¡SI NECESITO LOCALIZACION!





INTRODUCCIÓN A LA ROBÓTICA

Índice:

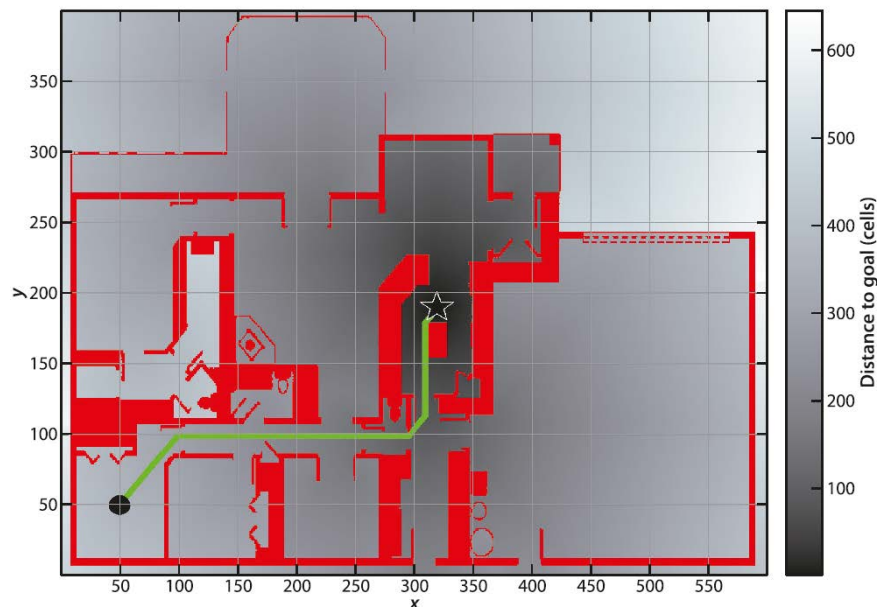
- 1 Navegación
- 2 Navegación sin mapa
- 3 Navegación con mapa
- 4 Navegación con mapa planificada**



IR: 4 – NAVEGACIÓN CON MAPA PLANIFICADA

Transformación de distancia.

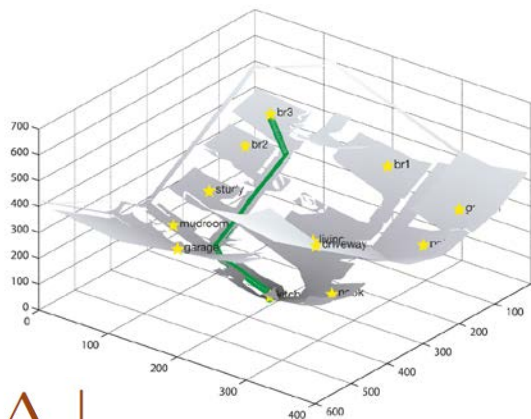
- Consiste en calcular la distancia al punto objetivo de cada punto del mapa.
- El cálculo de la distancia genera un gradiente que el robot puede seguir para converger al objetivo.
- **Camino optimo.** Ver que toma una visión más global que los algoritmos de tipo Bug, a costa de un mayor computo.
- La implementación es iterativa, cada iteración tiene un costo de $O(N^2)$ y el número de iteraciones es al menos $O(N)$, donde N es la dimensión del mapa.



$$\Delta_y = y_2 - y_1 \quad \Delta_x = x_2 - x_1$$

Distancia Euclídea $\sqrt{\Delta_x^2 + \Delta_y^2}$

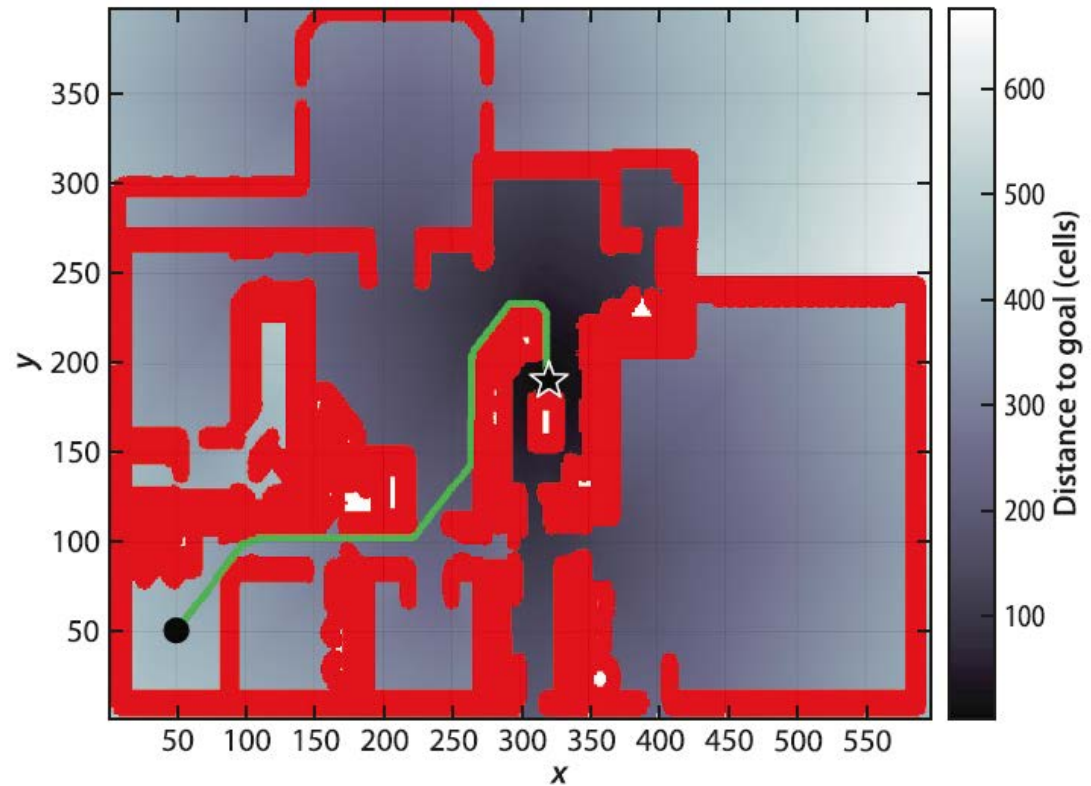
Distancia Manhattan (Cityblock) $|\Delta_x| + |\Delta_y|$



```
close all
clear all
% campo de obstaculos
load house
dx=DXform(house);
dx.plan(place.kitchen)
dx.plot();
dx.query(place.br3,'animate')
```

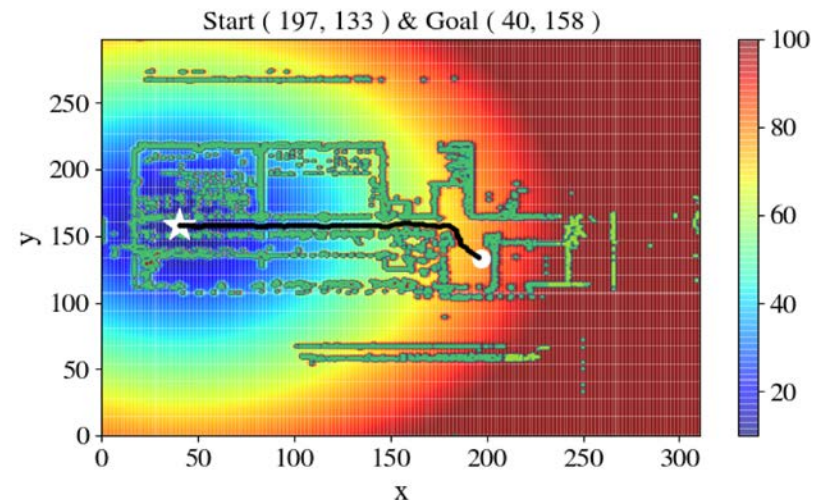
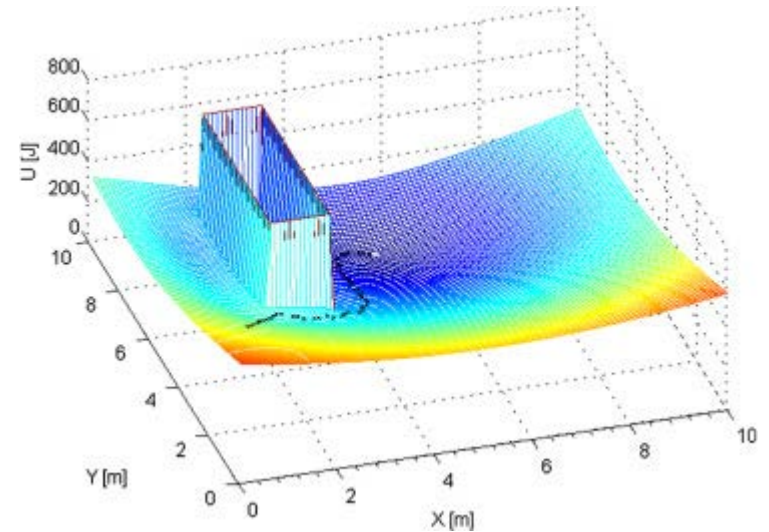
Transformación de distancia.

- Para grandes grillas de ocupación resolver el problema es impráctico (gran costo computacional).
- Otro problema (en realidad de la representación en grilla de ocupación), es que supusimos que el robot ocupa solo una casilla. Esto en general no es cierto, pero para que el planificador funcione podemos proponer “inflar” los obstáculos en el mapa.



Campos potenciales.

- Consiste en crear un campo, o gradiente, en el mapa de trabajo que direcciona al robot a la posición de destino.
- **Trata al robot como un punto bajo la influencia a de un campo potencial artificial $U(q)$.**
- El robot se mueve siguiendo el gradiente del campo potencial.
- La posición objetivo (un mínimo en el campo) actúa como una fuerza atractiva para el robot, mientras que los obstáculos como repulsivas.
- **No es solo un algoritmo de navegación sino una ley de control para el robot.**
- Si nuevos obstáculos aparecen durante le ejecución, es posible actualizar el campo potencial.
- **Es un método valido para sistemas de n dimensiones.**





Campos potenciales. Caso 2D

- En el caso más simple el robot se trata como un punto y la orientación θ se desprecia.
- Tomando un campo potencial $U(q)$ se puede definir una fuerza artificial $F(q)$ actuando en la posición $q = (x, y)$

$$F(q) = -\nabla U(q)$$

Donde $\nabla U(q)$ indica el vector gradiente de U en la posición q .

$$\nabla U = \begin{bmatrix} \frac{\partial U}{\partial x} \\ \frac{\partial U}{\partial y} \end{bmatrix}$$

- El campo potencial actuando sobre el robot se puede calcular como la suma del campo atractivo del objetivo y los campos repulsivos de los obstáculos.

$$U(q) = U_{att}(q) + U_{rep}(q)$$

- Y en forma similar las fuerzas producidas pueden separarse por ambas fuentes.

$$F(q) = F_{att}(q) - F_{rep}(q) = -\nabla U_{att}(q) - \nabla U_{rep}(q)$$



Campos potenciales. Caso 2D

- Un **campo potencial atractivo**, puede ser definido por ejemplo como una función parabólica:

$$U_{att}(q) = \frac{1}{2}k_{att} \cdot \rho_{goal}^2(q)$$

donde k_{att} es un factor de escala positivo y $\rho_{goal}(q)$ es la distancia Euclídea $\|q - q_{goal}\|$

- Este campo potencial es diferenciable, lo que nos permite obtener una fuerza F_{att} que converge linealmente hacia cero a medida que el robot tiende al objetivo.

$$\begin{aligned} F_{att}(q) &= -\nabla U_{att}(q) \\ &= -k_{att} \cdot \rho_{goal}(q) \nabla \rho_{goal}(q) \\ &= -k_{att} \cdot (q - q_{goal}) \end{aligned}$$



Campos potenciales. Caso 2D

- Un **campo potencial repulsivo**, por ejemplo, puede ser definido como:

$$U_{rep}(q) = \begin{cases} \frac{1}{2}k_{rep}\left(\frac{1}{\rho(q)} - \frac{1}{\rho_0}\right)^2 & \text{if } \rho(q) \leq \rho_0 \\ 0 & \text{if } \rho(q) \geq \rho_0 \end{cases}$$

donde k_{rep} es un factor de escala y $\rho(q)$ es la mínima distancia desde q al objeto y ρ_0 la distancia de influencia del objeto.

- El campo potencial repulsivo U_{rep} es positivo o cero y tiende a infinito a medida que q se acerca al objeto.
- Si el objeto tiene un perímetro convexo y por partes diferenciable, $\rho(q)$ es diferenciable en todo el espacio de configuración libre.
- Esto conduce a una fuerza repulsiva F_{rep} de la forma:

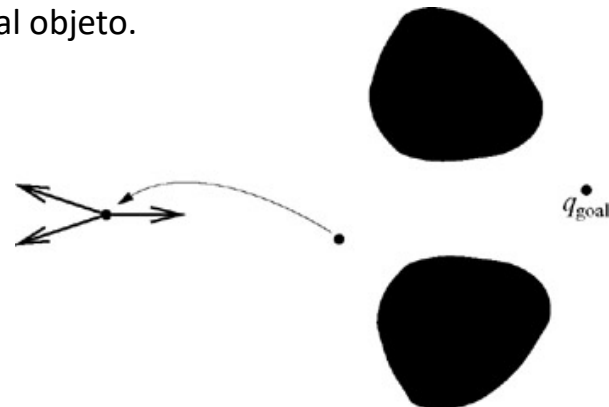
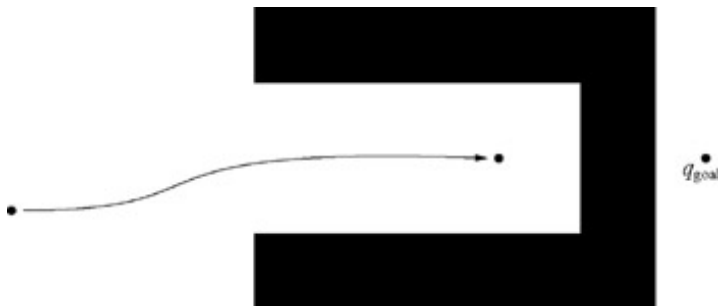
$$F_{rep}(q) = -\nabla U_{rep}(q) = \begin{cases} k_{rep}\left(\frac{1}{\rho(q)} - \frac{1}{\rho_0}\right)\frac{1}{\rho^2(q)}\frac{q - q_{obstacle}}{\rho(q)} & \text{if } \rho(q) \leq \rho_0 \\ 0 & \text{if } \rho(q) \geq \rho_0 \end{cases}$$

Campos potenciales. Caso 2D

- La fuerza resultante resulta ser:

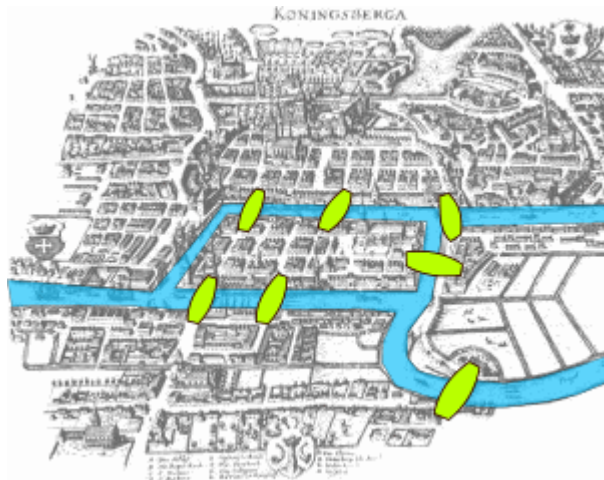
$$F(q) = F_{att}(q) + F_{rep}(q)$$

- Bajo las condiciones ideales, configurando el vector velocidad del robot proporcional al campo vector, el robot será guiado suavemente hacia el objetivo.
- Limitaciones:**
 - Mínimos locales, dependen de la forma y tamaño de los obstáculos, pudiendo generar condiciones de trampa.
 - Objetos cóncavos, pueden conducir a situaciones donde múltiples distancias mínimas existen, resultando en una oscilación entre los puntos cercanos al objeto.



Navegación basada grafos.

- La teoría de grafos tiene sus raíces en el campo matemático.
- Su utilización en el campo de la robótica se debe a la necesidad de algoritmos de tiempo real, que puedan trabajar con mapas cambiantes (grafos cambiantes).
- La mayoría de estos métodos distinguen dos pasos:
 - La **construcción del grafo**, donde los nodos son ubicados y unidos por vías o aristas.
 - La **búsqueda en el grafo**, donde la búsqueda de un camino (a veces optimo) entre dos nodos es realizada.

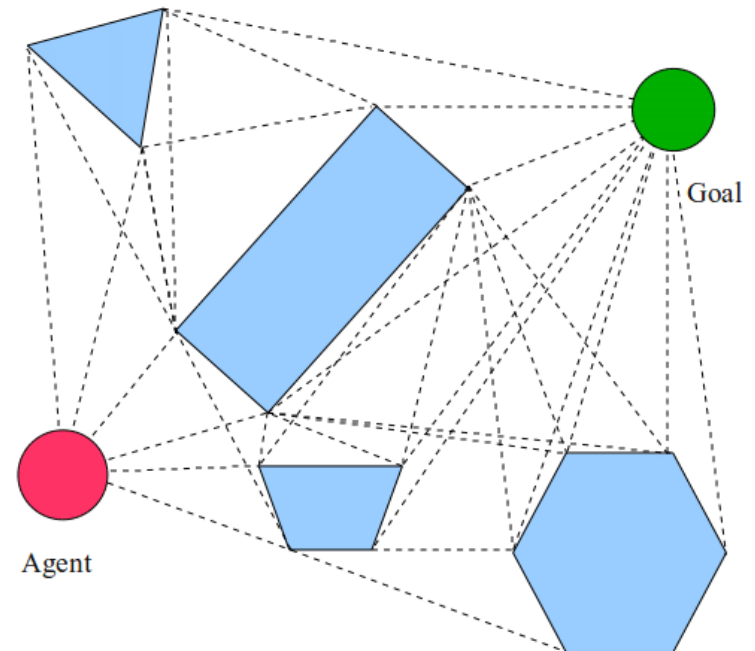


El problema de los **Siete Puentes de Königsberg**, considerado primer problema de grafos publicado. **Leonhard Euler**.

En la ciudad de Königsberg en Prusia (ahora Kaliningrado, Rusia) era una isla con algunos puentes, el problema era idear una caminata por la ciudad que cruzara cada uno de esos puentes una vez y solo una vez. Euler demostró que el problema no tiene solución.

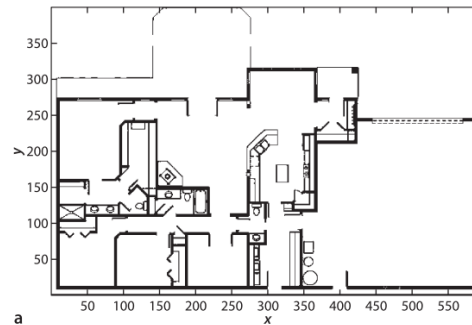
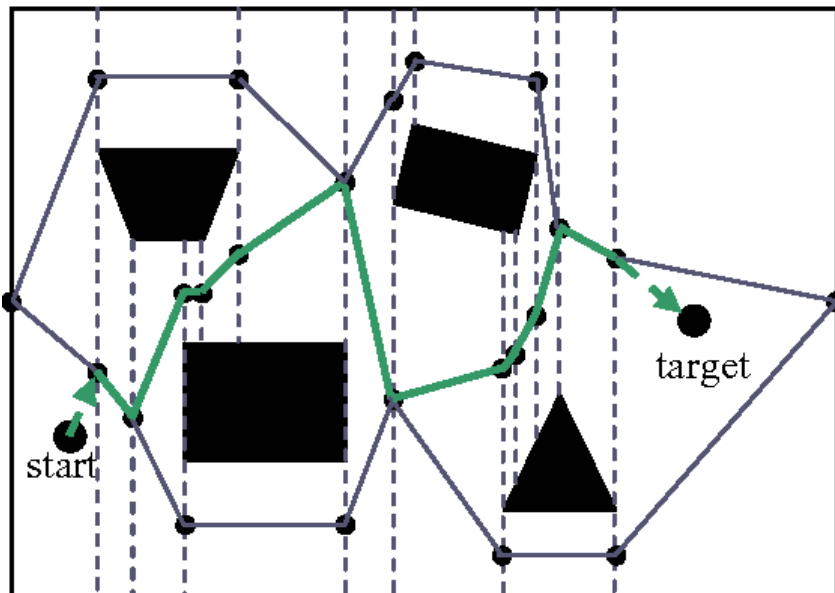
Construcción del grafo - Visibility graph (grafos de visibilidad).

- Para el caso de obstáculos polinomiales consiste en caminos que unen todos los vértices en el mapa.
- Las líneas rectas uniando estos vértices son las distancias más cortas, por lo que el planeador debe buscar la combinación de estas rectas que minimice la distancia total entre un punto inicial y uno objetivo.
- Los caminos encontrados son óptimos en términos de la longitud del camino.
- **Desventajas/advertencias:**
 - Número de líneas y nodos aumenta con el número de obstáculos en el mapa.
 - Las soluciones encontradas tienden a conducir al robot lo más cerca posible de los obstáculos.

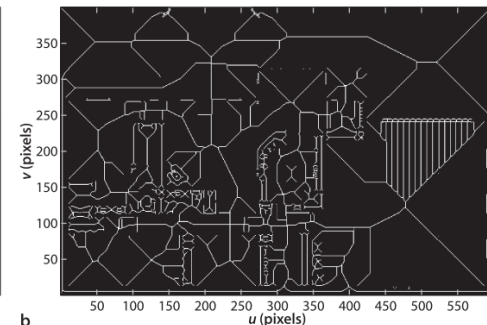


Construcción del grafo – Voronoi diagram

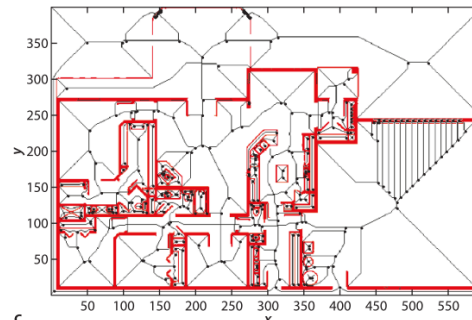
- Tiende a maximizar la distancia entre el robot y los obstáculos en el mapa.
- Consiste en la construcción de líneas formadas por los puntos equidistantes de dos o más obstáculos.
- **Desventajas/advertencias:**
 - Los caminos solución usualmente se encuentran lejos del camino optimo en el sentido de distancia.
 - Si se cuenta con sensores de corto alcance en el robot, por el tipo de recorrido generado pueden que no logren medir el ambiente de trabajo.



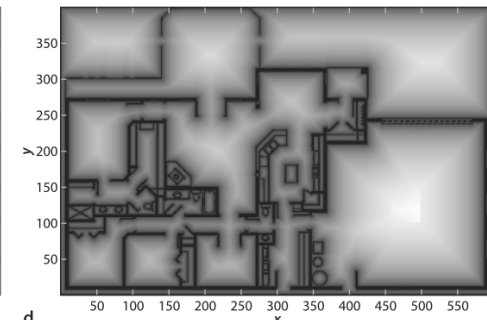
a



b



c

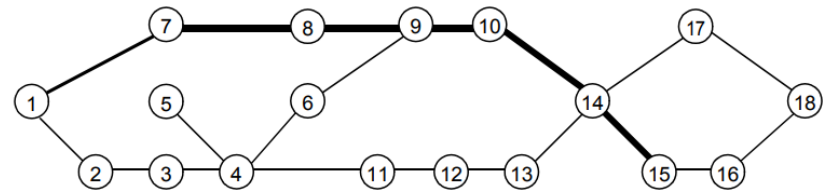
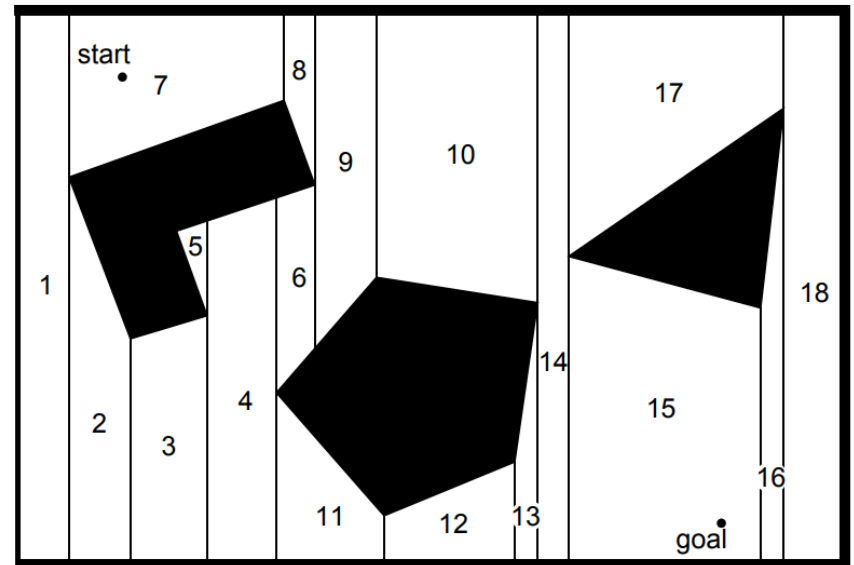


d



Construcción del grafo – Exact cell decomposition

- Identificación geométrica de celdas en el mapa.
- La posición del robot en cada celda no es lo importante, sino la capacidad de moverse de una celda a la adyacente.
- Si el ambiente no tiene gran cantidad de elementos el número de celdas es bajo, independientemente del área real del mapa.
- **Desventajas/advertencias:**
 - Si el mapa es denso en el número de objetos se generan un gran número de celdas, computacionalmente más demandante.
 - **Todas las versiones de descomposición en celdas pueden representarse como grafos, incluidas las grillas de ocupación.**



Poco usada en robótica por complejidad de implementación

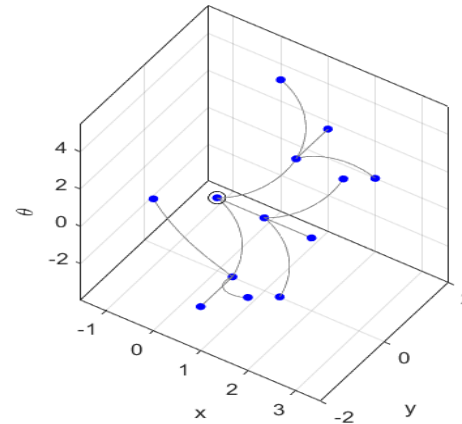
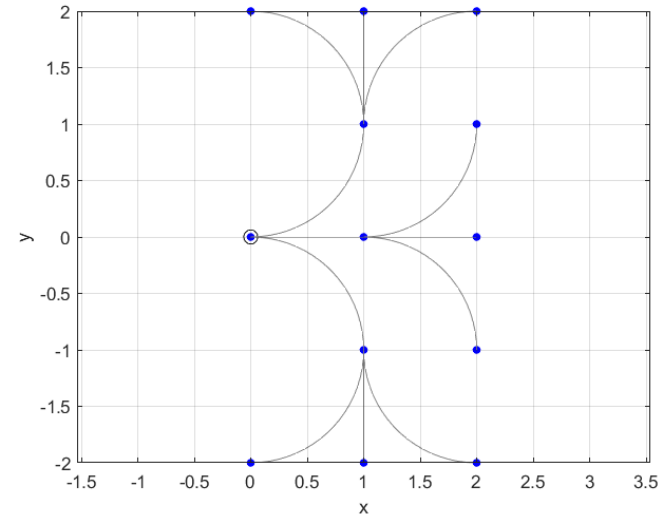


IR: 4 – NAVEGACIÓN CON MAPA PLANIFICADA

Construcción del grafo – Lattice graph

- Se construyen a partir de un conjunto de líneas que representan los posibles movimientos de un determinado robot. Luego a partir de los puntos alcanzados se vuelven a proyectar estas posibles trayectorias hasta completar todo el espacio libre en el mapa.
- La principal ventaja es que considera caminos realizables por el robot ya que considera su cinemática.

```
close all  
clear all  
lp=Lattice();  
lp.plan('iterations',2);  
lp.plot();
```





IR: 4 – NAVEGACIÓN CON MAPA PLANIFICADA

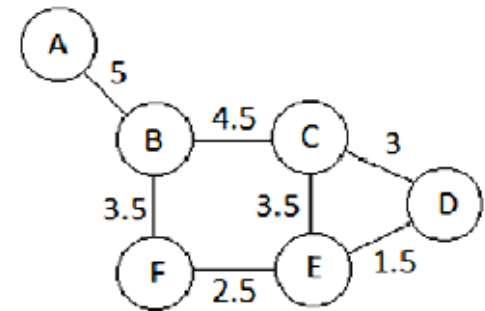
Búsqueda en el grafo (métodos determinísticos)

- El objetivo de la planificación de camino es encontrar el mejor camino en el grafo que representa el mapa real entre un punto de inicio y el objetivo. El término “mejor” hace referencia a un criterio de optimización (por ej. camino más corto, más rápido, menor consumo de energía, etc.)
- En forma genérica todos los algoritmos de búsqueda tienen algún tipo de discriminador de la siguiente forma:

$$f(n) = g(n) + \varepsilon \cdot h(n)$$

Donde:

- $f(n)$ es el costo total esperado
- $g(n)$ es el costo acumulativo del camino, que considera cada costo individual de travesar caminos entre dos nodos n y n' (costo de vía) $c(n, n')$
- Costo heurístico $h(n)$ (función que aporta alguna información extra sobre el mapa)
- ε es un parámetro que determina el grado de dependencia con la heurística.



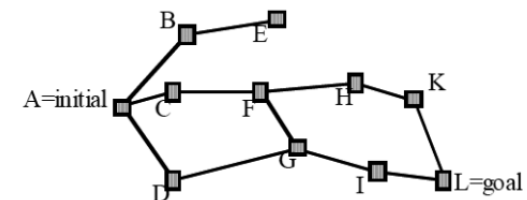
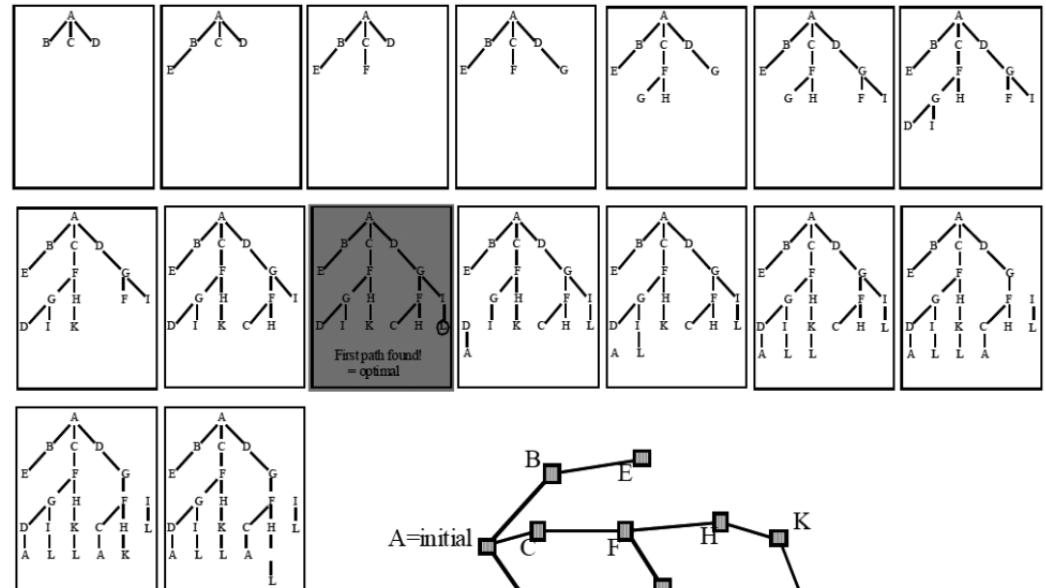
A							
0							
5							
B	0	4.5	C	0	3	D	0
3.5			3.5			1.5	
F	0	2.5	E	0	0	0	



IR: 4 – NAVEGACIÓN CON MAPA PLANIFICADA

Búsqueda en anchura (Breadth-first search)

- Este método de búsqueda inicia en un nodo y luego explora todos sus nodos vecinos. Luego, para cada uno de estos nodos explora los vecinos inexplorados y de esta forma continua.
- Los nodos son expandidos en orden de proximidad – tomándola como el menor número de transiciones de aristas desde el nodo inicial.
- Este método **siempre devuelve el camino con menos transiciones** de aristas desde el inicio hasta el punto objetivo.
- Si el costo de cada arista es constante en todo el grafico, el camino encontrado es óptimo.
- El proceso se lo denomina **expansión de nodo**:
 - Un nodo esta **activo** cuando se exploran sus vecinos
 - Al ser explorado se lo marca como **abierto**
 - Cuando se concluye la exploración se lo define como **visitado**.

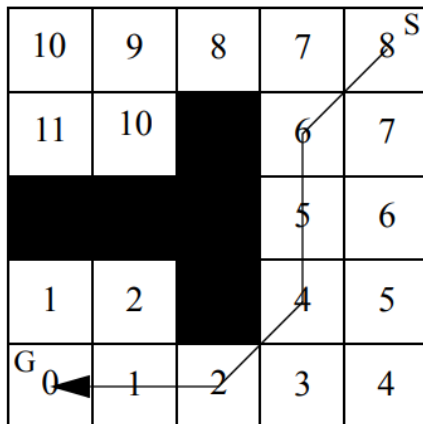




IR: 4 – NAVEGACIÓN CON MAPA PLANIFICADA

Wavefront expansión algorithm

- Es un caso particular de búsqueda en anchura.
- Este método utiliza una grilla de ocupación como mapa.
- El algoritmo emplea la expansión del frente de onda desde la posición objetivo hacia afuera, marcando para cada celda su distancia (Manhattan) a la celda objetivo. El proceso continua hasta que la celda que contiene al robot es alcanzada.
- El planificador de camino puede estimar la distancia del robot a la posición de destino, así como recuperar una trayectoria de solución específica simplemente uniendo celdas que son adyacentes y siempre más cerca de la meta.
- Cada celda solo se visita una vez cuando se busca para la ruta discreta más corta desde la posición inicial hasta la posición de destino, por tanto la complejidad de computo solo es lineal con el número de celdas.



obstacle cell



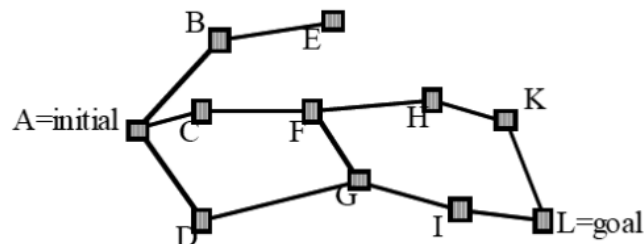
cell with distance value

WAVEFRONT PROPAGATION ALGORITHM

1. Initialize $W_0 = X_G$; $i = 0$.
2. Initialize $W_{i+1} = \emptyset$.
3. For every $x \in W_i$, assign $\phi(x) = i$ and insert all unexplored neighbors of x into W_{i+1} .
4. If $W_{i+1} = \emptyset$, then terminate; otherwise, let $i := i + 1$ and go to Step 2.

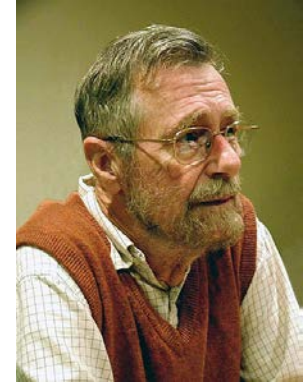


- La búsqueda en profundidad expande cada nodo hasta el nivel más profundo del grafo (hasta que el nodo no tenga más sucesores).
- Como esos nodos son expandido, su rama se elimina del grafo y la búsqueda retrocede expandiendo el siguiente nodo vecino del nodo de inicio hasta su nivel más profundo y así sucesivamente.
- Un inconveniente de este algoritmo es que puede volver a visitar nodos previamente visitados o encontrar caminos redundantes.



Algoritmo de Dijkstra

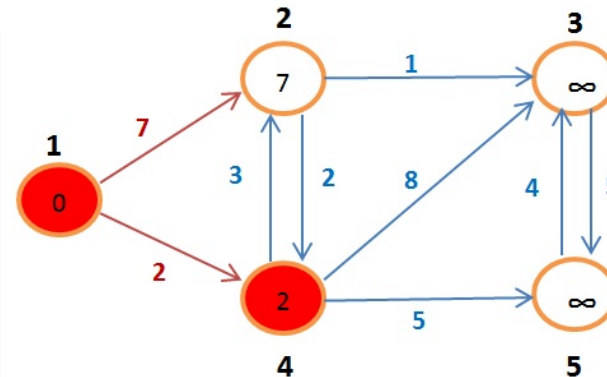
- Similar al algoritmo de búsqueda en anchura, con la excepción que el coste de arista puede ser cualquier valor positivo y la solución garantiza un camino optimo.
- En este algoritmo los nodos a explorar son ordenados en una cola de prioridad de forma que el siguiente nodo a explorar siempre es aquel que tiene el menor costo total de camino (tomado desde los elementos de la cola).
- El proceso inicia en un nodo, se arma la “cola de prioridad” de costos, se explora el siguiente nodo con menor costo, se actualiza la “cola” con los vecinos del nuevo nodo, se sigue la exploración por el de menor costo...
- El proceso termina cuando se arriba al nodo objetivo o cuando no hay más nodos a explorar en la “cola”.



Edsger Dijkstra en 2002

```

DIJKSTRA (Grafo  $G$ , nodo_fuente  $s$ )
  para  $u \in V[G]$  hacer
    distancia[ $u$ ] = INFINITO
    padre[ $u$ ] = NULL
  distancia[ $s$ ] = 0
  Encolar (cola, grafo)
  mientras que cola no es vacía hacer
     $u$  = extraer_minimo(cola)
    para  $v \in \text{adyacencia}[u]$  hacer
      si distancia[ $v$ ] > distancia[ $u$ ] + peso ( $u$ ,  $v$ ) hacer
        distancia[ $v$ ] = distancia[ $u$ ] + peso ( $u$ ,  $v$ )
        padre[ $v$ ] =  $u$ 
        Actualizar(cola, distancia,  $v$ )
  
```



Cola	
Nodo	Peso
4	2
2	7

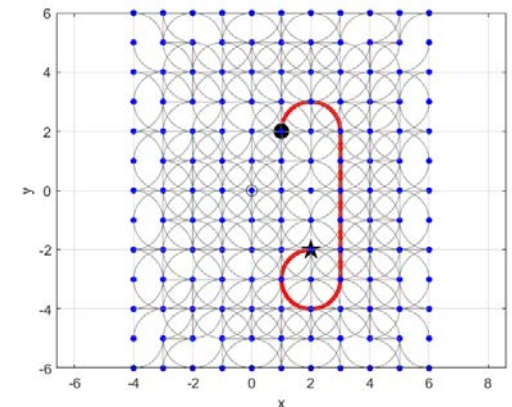
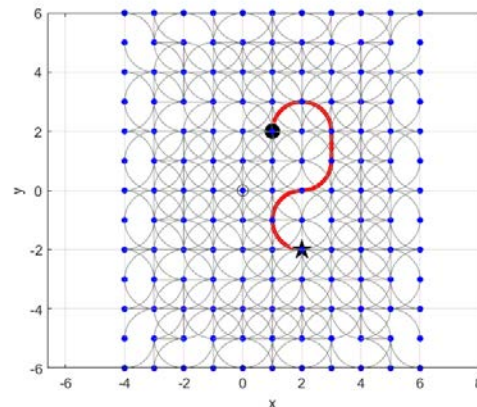


IR: 4 – NAVEGACIÓN CON MAPA PLANIFICADA

Algoritmo A* (algoritmo A estrella)

- Similar al algoritmo de Dijkstra, solo que incluye una función heurística $h(n)$ que agrega conocimiento adicional del grafo.
- En robótica A* es mayormente empleado en grillas de ocupación y la heurística tomada suele ser la distancia entre la celda en consideración y una línea recta a la celda objetivo. -> En comparación con Dijkstra, reduce el número de expansión de nodos para llegar a un resultado. (“**dirige la conversión del algoritmo**”)
- La principal diferencia es que el orden de la cola de extracción es en base al nodo con menor coste total de camino $f(n) = g(n) + \varepsilon \cdot h(n)$
- Cuando ε es uno se busca la solución óptima, si hacemos ε mayor a 1 se buscan soluciones subóptimas, que favorecen la velocidad del algoritmo.

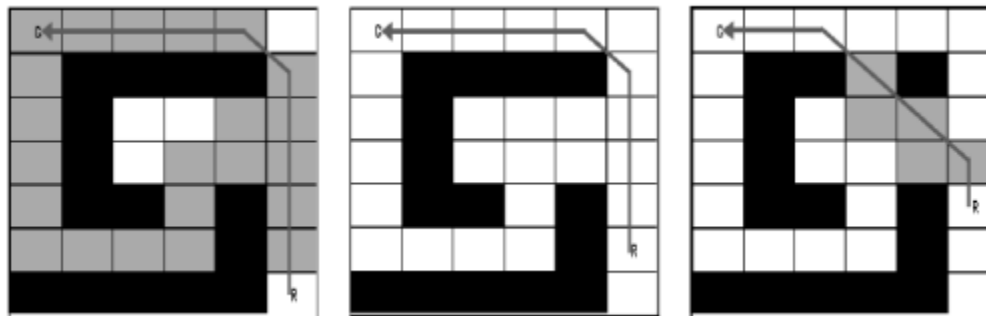
```
close all
clear all
start=[1 2 pi/2] %punto inicial
goal=[2 -2 0] %punto objetivo
lp=Lattice(); % representacion en
diagram lettice de robot tipo bicicleta con
tres acciones posibles
%lp.plan('iterations',6,'cost',[1 1 1]);
%numero de iteraciones y costo de
%acciones "ir derecho", "doblar izq",
"doblar der"
lp.plan('iterations',6,'cost',[1 100 1]);
lp.query(start,goal);
```





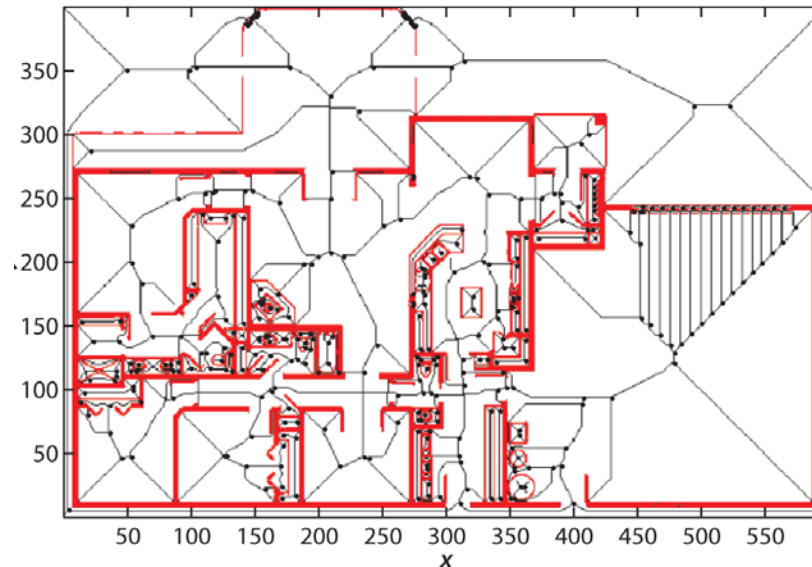
Algoritmo D* (algoritmo D estrella)

- Similar al algoritmo A*, pero permite la replanificación parcial en función de elementos que cambiaron localmente en el mapa (por ejemplo, debido a las lecturas realizadas en el propio robot).
- Funcionamiento:
 - A partir de un mapa se obtiene el camino utilizando el algoritmo A*.
 - Luego de un tiempo de recorrer el camino el robot actualiza el mapa con medidas de sus sensores. -> se necesita una nueva planificación.
 - Solo se recomputan los estados agregados o removidos del mapa, usualmente el nuevo cálculo parte desde la posición objetivo por tanto gran parte de la solución previa se mantiene intacta.
- Comparado con A* el tiempo de búsqueda decrece en un factor de 1 a 2 órdenes.



Métodos de hojas de ruta (roadmaps)

- Métodos analizados hasta el momento requieren tiempo para generar la fase de planificación, mientras que la etapa de búsqueda es relativamente rápida.
- Algunos de los métodos permiten ajustar el mapa ante eventuales modificaciones, pero no permiten cambiar el destino sin tener que replanificar.
- Lo anterior es una justificación para los métodos denominados “hojas de ruta” (**roadmaps**), los cuales permiten computarse una única vez y luego son usados como una red de caminos (los diagramas de Voronoi son un ejemplo de esta idea).

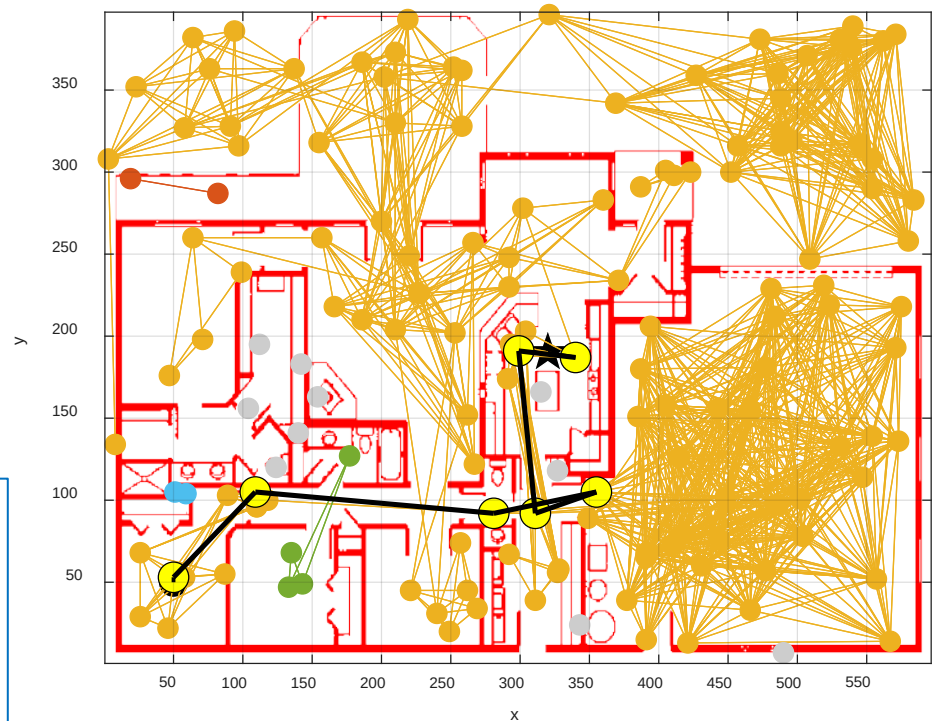


Métodos de hojas de ruta probabilísticos (PRM)

- En situaciones con una alta complejidad del espacio de trabajo, o donde los tiempos de cómputo son elevados o cuando no es posible encontrar una función heurística apropiada -> surgen los **métodos de hoja de ruta probabilísticos** (Probabilistic Roadmap Method - PRM).
- El más conocido es el método PRM.
 - Este a partir de un mapa conocido crea una cantidad de nodos n de posición aleatoria.
 - Luego conecta todos los nodos posibles por líneas rectas que no crucen ningún obstáculo.
 - Luego para desplazarse en el grafo generado se puede recurrir a cualquier método de búsqueda por ejemplo A* y tomar los nodos más cercanos al punto de inicio y objetivo.

```
clear all
close all
load house
prm=PRM(house)
prm.plan('npoints',150)
prm.plot()

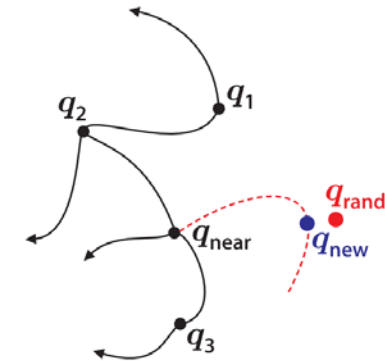
prm.query(place.br3, place.kitchen);
prm.plot()
```



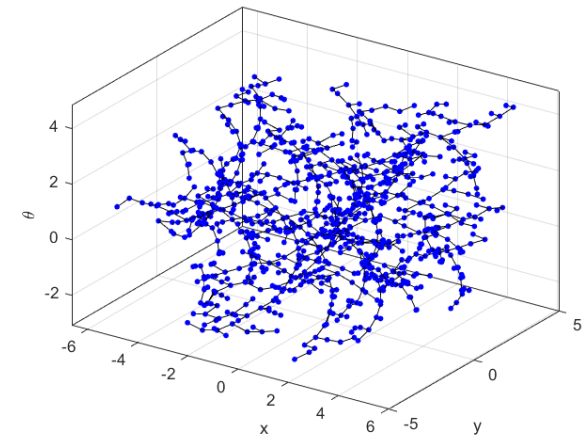


Rapidly-Exploring Random Tree (RRT)

- Este método puede verse como una combinación del planificador Lattice, ya que considera el modelo del robot en cuestión, y el método PRM, ya que toma n configuraciones aleatoria en el espacio de trabajo a fin de generar un “árbol” de posibles acciones.
- Los pasos en la creación de un RRT son los siguientes.
 - Se parte de un nodo inicial en el grafo (puede ser la posición inicial).
 - Se toma una configuración aleatoria q_{rand} .
 - Se selecciona el punto realizable más cercano a q_{near} perteneciente al grafo (al decir cercano evaluamos una función de costo que incluye distancia y orientación)
 - Se computa un control que mueva el robot al nuevo punto y se agrega al grafo q_{new} (si no cae sobre un obstáculo)
 - Se repite el ciclo, hasta fin iteraciones o error menor a cierto margen apunto destino.



```
close all
clear all
car=Bicycle('steermax',0.5);
rrt=RRT(car,'npoints',1000)
rrt.plan();
rrt.plot();
```





INTRODUCCIÓN A LA ROBÓTICA

Lo visto...

- Navegación.
- Campos potenciales.
- Construcción de grafos.
- Navegación mediante grafos.
- Métodos probabilísticos.

Próxima clase:

- Técnicas de evitación de obstáculos.

Bibliografía:

- Robotics, Vision and Control Fundamental Algorithms in MATLAB. Peter Corke. 2da edición Springer.
- Howie Choset. Principles of robot motion: Theory, Algorithms, and Implementation.
- Introduction to Autonomous Mobile Robot. Roland Siegwart. 2da edición. MIT Press.

